ARCHITECTPOINT · BACKEND DEEP DIVE

Spring Security & JWT — Zero se Samjho

Config ki saari 7 files, request ka pura safar, aur wo traps jo 403 ko 401 bana dete hain — Hinglish me, examples ke saath.

Stack: Spring Boot 3.0.1 · Spring Security 6 · Java 17 · JWT (jjwt) · MySQL `appdb`
Covers: SecurityConfig · JwtUtils · JwtAuthenticationFilter · MyUserDetailsService · UserPrinciple · JwtAuthenticationEntryPoint · CrosConfig

PART 0 — FOUNDATION

Security ka Basic: Do Alag Sawaal

> Security me sabse pehla confusion yahi hota hai ki "login" aur "permission" ek hi cheez lagti hai. Ye do bilkul alag cheezein hain, aur poora SecurityConfig inhi dono ke upar khada hai.

Authentication vs Authorization

	Authentication	Authorization
Sawaal	**"Tum kaun ho?"**	**"Tum ye kaam kar sakte ho?"**
Kab hota hai	Pehle — identity verify	Baad me — permission check
Is project me	JWT token valid hai? Us token ka user DB me hai?	Us user ka role `Admin` hai ya `User`? Ye endpoint uske role ko allow karta hai?
Fail hone pe	**401 Unauthorized** — "pehchaan hi nahi hui"	**403 Forbidden** — "pehchaan to ho gayi, par permission nahi"
Kaun handle karta hai	`JwtAuthenticationFilter` + `JwtAuthenticationEntryPoint`	`SecurityConfig` ke URL matchers

> **Real-life analogy — Office building**
>
> **Authentication** = gate pe ID card dikhana. Guard confirm karta hai "haan, ye Prashant hi hai, hamari company ka employee hai."
> **Authorization** = us ID card pe likha hai "Floor 3 access". Ab tum building ke andar to aa gaye (authenticated), par agar Floor 7 (server room) jaane ki koshish karoge to lift tumhe rok degi (403) — kyunki tum *ho to sahi insaan*, par *us floor ki permission nahi hai*.
>
> Agar tum bina ID card ke aaye — guard gate pe hi rok dega (401). Andar ghusne ka sawaal hi nahi.

Password kabhi plain text me store nahi hota

DB me password aise store hota hai:

```
password: $2a$10$N9qo8uLOickgx2ZMRZoMye1J/8f4pQ.qPLqK6m...   // BCrypt hash
// NOT:  password: test123   ← ye kabhi mat karna
```

BCrypt one-way hashing hai — hash se wapas original password nikaalna practically impossible hai. To login ke time compare kaise karte hain? Ulta karte hain: jo password user ne type kiya, usko *same algorithm se hash karke* DB wale hash se match karte hain.

```
passwordEncoder.matches("test123", "$2a$10$N9qo8...")  // → true / false
//                      ↑ user ne jo type kiya     ↑ DB me jo stored hai
```

Ye `passwordEncoder` bean kahan se aata hai? `SecurityConfig` se — wahi ek line jo poore app me password hashing ka standard set karti hai:

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

Kyun zaroori hai

Agar kal ko tumhara DB leak ho jaaye, hacker ko sirf hashes milenge — actual passwords nahi. Users ne jo same password doosri sites pe use kiya hai, wo bhi safe rehta hai. Isi liye `@JsonProperty(WRITE_ONLY)` bhi lagaya hai `User.password` pe — hash bhi API response me kabhi leak na ho.

PART 1 — JWT

JWT Token: Kya hai aur Kyun

> Purane zamane me "session" hota tha — server har logged-in user ki memory rakhta tha. JWT ka pura funda ulta hai: server kuch yaad nahi rakhta, saari information token ke andar hi encode hoti hai.

Session vs JWT (Stateless)

	Session-based (purana)	JWT (is project me)
Server kya store karta hai	Har logged-in user ka session memory/DB me	**Kuch bhi nahi** — stateless
Client ke paas kya hai	Session ID (ek random string)	Pura signed token (identity + role andar hai)
Verify kaise hota hai	Session ID se server apni memory check karta hai	Token ka **signature** verify hota hai — DB/memory lookup nahi
Server restart pe	Sab log out ho jaate hain (memory gayi)	Token abhi bhi valid — kuch farak nahi

Isi stateless behaviour ko `SecurityConfig` me ye line enforce karti hai:

```
.sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

Matlab: "Spring, tum session banana hi mat. Har request ko bilkul naya samajhna — jo bhi identity chahiye, wo request ke token se aayegi."

Token ke andar kya hota hai

JWT 3 hisso me bata hota hai, dot se separated:

```
eyJhbGciOiJIUzI1NiJ9 . eyJyb2xlIjoiQWRtaW4iLCJzdWIiOiJhZG1pbkB0ZXN0LmNvbSJ9 . 4f2Xq9...
└──── HEADER ────┘   └──────────────── PAYLOAD ────────────────┘   └─ SIGNATURE ─┘
   algorithm: HS256      role: "Admin", sub: "admin@test.com",       secret key se
                         iat: issued-at, exp: expiry                 banaya hua hash
```

Important: Payload encrypted NAHI hai

Wo bada sa string dekh ke lagta hai "encrypted hai" — par wo sirf **Base64 encoded** hai. Koi bhi usko decode karke andar ka role/email padh sakta hai (jwt.io pe paste karke dekh lo).

To fir security kahan hai? **SIGNATURE me.** Agar koi payload me `"role": "User"` ko `"role": "Admin"` kar de, to signature match nahi karega (kyunki signature secret key se bana tha), aur `validateToken()` false return kar dega. Matlab: **JWT me data chhupa nahi hota — bas tamper-proof hota hai.** Isiliye password jaisi cheez kabhi token ke andar mat daalna.

Tumhare project ka token kaise banta hai

```java
// JwtUtils.java
private static final String SECRET = "architectPointSecretKeyForJwtSigning2025";
private static final long EXPIRATION_MS = 24 * 60 * 60 * 1000L;   // 24 ghante

public String generateToken(String email, String userRole) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("role", userRole);          // ← role token me ghusa diya
    return Jwts.builder()
            .setClaims(claims)
            .setSubject(email)               // ← "sub" = user ki pehchaan (email)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + EXPIRATION_MS))
            .signWith(SignatureAlgorithm.HS256, SECRET)   // ← yahi tamper-proof banata hai
            .compact();
}
```

Project trap — role zaroor DB se aana chahiye

Dekho `generateToken(email, userRole)` — ye `userRole` **DB row se** aata hai (`user.getUserRole()`), hardcode nahi hai. Agar yahan role blind token banaya (ya dono tokens me same role chala gaya), to login ke dono tests to pass ho jaayenge, par baad ke **6 tests** bilkul unrelated dikhne wale reasons se fail honge — kyunki Admin aur User dono ko same permissions mil jaayengi.

PART 2 — FILE BY FILE

Saari 7 Files ka Kaam

> Har file ka ek hi, clearly defined kaam hai. Pehle poora map dekh lo, fir ek-ek karke detail me jaayenge.

File	Ek line me kaam
`SecurityConfig.java`	**Rule book** — kaunsa URL kaun access kar sakta hai, aur poori chain wire karta hai
`JwtUtils.java`	**Token factory** — token banata hai, padhta hai, validate karta hai
`JwtAuthenticationFilter.java`	**Gatekeeper** — har request pe token check karke identity set karta hai
`MyUserDetailsService.java`	**DB lookup** — email se user ka record nikaalta hai
`UserPrinciple.java`	**Translator** — tumhara `User` → Spring Security ka format
`JwtAuthenticationEntryPoint.java`	**401 bhejne wala** — jab koi bina valid token ke aaye
`CrosConfig.java`	**Browser gate** — kaunsa frontend origin API call kar sakta hai

`JwtUtils.java` — Token banao, padho, validate karo

4 methods hain, saare ek hi `SECRET` key ke around ghoomte hain:

Method	Kya karta hai
`generateToken()`	Login ke time naya token banata hai — email + role andar daal ke, secret se sign karke
`extractUsername()`	Token se `sub` (email) nikaalta hai — "ye token kiska hai?"
`extractRole()`	Token se `role` claim nikaalta hai
`validateToken()`	Signature + expiry check — tampered ya expired token pe `false`

```java
public boolean validateToken(String token) {
    try {
        Jwts.parser().setSigningKey(SECRET).parseClaimsJws(token);
        return true;      // parse ho gaya = signature sahi + expire nahi hua
    } catch (Exception e) {
        return false;     // tampered / expired / garbage → sab yahan
    }
}
```

Dhyan do — ye **DB ko touch hi nahi karta**. Sirf maths hai: "is token ka signature mere secret se banta hai ya nahi?" Yahi stateless ka asli matlab hai.

`UserPrinciple.java` — User → Spring Security format

Spring Security tumhare `User` entity ko nahi samajhta — wo sirf `UserDetails` interface samajhta hai. Ye class **adapter** ka kaam karti hai: tumhare `User` ko wrap karke Spring ke format me pesh karti hai.

```
@Override
public Collection<? extends GrantedAuthority> getAuthorities() {
    return List.of(new SimpleGrantedAuthority("ROLE_" + user.getUserRole()));
}                                                  // ↑ ye prefix CRITICAL hai

@Override
public String getUsername() { return user.getEmail(); }  // Spring ke liye "username" = email
```

Sabse bada gotcha — `ROLE_` prefix

DB me role store hai: `"Admin"`.
Yahan authority banti hai: `"ROLE_Admin"`.
SecurityConfig me likha hai: `.hasRole("Admin")`.

Ye teeno match kaise karte hain? Kyunki `hasRole("Admin")` internally khud `"ROLE_"` prefix lagata hai → wo `"ROLE_Admin"` dhundhta hai. Isliye tumhe `UserPrinciple` me manually `"ROLE_"` lagana **padta** hai. Agar ye prefix hata do, har role check fail ho jaayega aur sab 403 milega.

Aur case-sensitive hai! `"ROLE_Admin"` ≠ `"ROLE_ADMIN"`. DB me `"Admin"` hai (capital A, baaki small) — to config me bhi bilkul `hasRole("Admin")` hi likhna, `hasRole("ADMIN")` nahi.

`MyUserDetailsService.java` — Email se user DB se laao

```
@Override
public UserDetails loadUserByUsername(String email) {
    User user = userRepo.findByEmail(email)
            .orElseThrow(() -> new UsernameNotFoundException("User not found: " + email));
    return new UserPrinciple(user);     // ← wrap karke Spring ko de diya
}
```

Chhota sa code, par important role: token me sirf **email** hota hai. Us email se actual DB record (aur uska role) nikaalna — yahi iska kaam hai. Phir `UserPrinciple` me wrap karke Spring Security ko handover.

Kyun DB se dobara padhte hain jab role token me already hai? Security ke liye. Agar admin ne kisi user ka role DB me change kar diya, ya user delete kar diya — to purana token hone ke bawajood wo access nahi kar payega, kyunki har request pe fresh DB lookup hota hai.

`JwtAuthenticationFilter.java` — Har request ka gatekeeper

Ye poore system ka **dil** hai. `OncePerRequestFilter` extend karta hai — matlab har single HTTP request pe exactly ek baar chalta hai, controller tak pahunchne se **pehle**.

```
String header = request.getHeader("Authorization");

if (header != null && header.startsWith("Bearer ")) {
    String token = header.substring(7);          // "Bearer " (7 chars) hata do
    try {
        if (jwtUtils.validateToken(token)) {         // 1. signature sahi hai?
            String email = jwtUtils.extractUsername(token);          // 2. kiska token?
            if (email != null && SecurityContextHolder.getContext()
                                    .getAuthentication() == null) {
                UserDetails userDetails =
                    userDetailsService.loadUserByUsername(email);    // 3. DB se user

                UsernamePasswordAuthenticationToken auth =
                    new UsernamePasswordAuthenticationToken(
                        userDetails, null, userDetails.getAuthorities());  // 4. authorities!

                auth.setDetails(new WebAuthenticationDetailsSource()
                                .buildDetails(request));
                SecurityContextHolder.getContext().setAuthentication(auth);  // 5. ★ set
            }
        }
    } catch (Exception e) {
        SecurityContextHolder.clearContext();     // kuch bhi gadbad → identity hatao
    }
}
filterChain.doFilter(request, response);          // aage badho (hamesha!)
```

Line 5 hi sabse important hai

`SecurityContextHolder.getContext().setAuthentication(auth)` — ye line Spring Security ko batati hai **"is request ka banda authenticated hai, aur ye uske roles hain."** Iske baad hi authorization (role check) ka sawaal aata hai.

> **Project trap — 401 vs 403 ka asli raaz**
>
> Agar ye filter `SecurityContext` set karne me fail ho jaaye, to Spring ko lagta hai "koi anonymous banda aaya hai" → **401** milega.
> Par tumhare 11 role-based tests me **valid token galat role ke saath** bheja jaata hai — unhe **403** expect hai, 401 nahi.
>
> Matlab: **agar tumhe har jagah 401 aa raha hai 403 ki jagah, to filter ka issue hai — SecurityConfig ka nahi.** Ye debugging shortcut yaad rakhna, bahut time bachayega.

Ek aur cheez notice karo — `filterChain.doFilter()` **hamesha** call hota hai, chahe token ho ya na ho. Filter kabhi request ko block nahi karta. Uska kaam sirf identity set karna hai; block/allow ka faisla aage `SecurityConfig` ke rules karte hain.

`JwtAuthenticationEntryPoint.java` — 401 bhejne wala

```java
@Override
public void commence(HttpServletRequest request, HttpServletResponse response,
                     AuthenticationException authException) throws IOException {
    response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Unauthorized");  // 401
}
```

Jab koi **bina token** (ya invalid token ke saath) protected endpoint pe aata hai, Spring Security iska `commence()` call karti hai. Ye wahi response hai jo tumne Thunder Client me dekha tha jab `/api/user/11` hit kiya tha bina token ke.

PART 3 — SECURITYCONFIG

Rule Book: Kaun Kya Access Kar Sakta Hai

> Ye file do kaam karti hai — (1) chain ka behaviour set karna, (2) URL-wise permissions declare karna.

Upar ka setup — chain ka behaviour

```
http
  .csrf(csrf -> csrf.disable())                // 1
  .cors(cors -> {})                            // 2
  .exceptionHandling(e -> e.authenticationEntryPoint(jwtAuthenticationEntryPoint))  // 3
  .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // 4
  .authorizeHttpRequests(auth -> auth ... )      // 5 — rules (neeche)
  .addFilterBefore(jwtAuthenticationFilter,
                   UsernamePasswordAuthenticationFilter.class);   // 6
```

#	Line	Kyun
1	`csrf.disable()`	CSRF attack cookies pe depend karta hai. Hum cookies use hi nahi karte (JWT header me aata hai), to ye protection yahan bematlab hai — aur enabled chhodo to har POST/PUT block ho jaayega.
2	`.cors(cors -> {})`	"CORS enable karo, par settings `CrosConfig` se uthao." Ye empty lambda matlab default CORS config use karna.
3	`authenticationEntryPoint`	Authentication fail hone pe kya karna hai — hamara custom 401 bhejne wala class.
4	`STATELESS`	Koi HTTP session mat banao. Har request independent — identity sirf token se.
6	`addFilterBefore(...)`	Hamara JWT filter Spring ke default username/password filter se **pehle** chale — taaki identity pehle set ho jaaye.

Asli rules — URL matchers

```
// ---- public ----
.requestMatchers("/api/register", "/api/login").permitAll()
.requestMatchers("/error").permitAll()

// ---- architects ----
.requestMatchers(POST,   "/api/architects").hasRole("Admin")
.requestMatchers(GET,    "/api/architects").authenticated()      // ← dono roles
.requestMatchers(GET,    "/api/architects/*").hasRole("Admin")
.requestMatchers(PUT,    "/api/architects/*").hasRole("Admin")
.requestMatchers(DELETE, "/api/architects/*").hasRole("Admin")

// ---- inquiries (most specific FIRST) ----
.requestMatchers(GET,    "/api/inquiries/user/*").hasRole("User")   // ← pehle!
.requestMatchers(POST,   "/api/inquiries").hasRole("User")
.requestMatchers(GET,    "/api/inquiries").hasRole("Admin")
...
.anyRequest().authenticated()
```

Trap 1 — Order matter karta hai (first match wins)

Spring Security rules ko **upar se neeche** padhti hai aur **pehla match jo mile wahi lagta hai**. Baaki rules dekhti hi nahi.

Isiliye `/api/inquiries/user/*` ko `/api/inquiries/*` se **pehle** likhna padta hai. Agar ulta likh diya, to `GET /api/inquiries/user/2` pehle broader rule se match ho jaayega aur galat permission lag jaayegi.

Trap 2 — Single `*` use karo, `**` nahi

`*` = sirf **ek** path segment match karta hai → `/api/architects/5` ✓ par `/api/architects/5/xyz` ✗
`**` = **saare** nested segments match karta hai → dono ✓

Agar tumne `/api/architects/**` likh diya, to wo `/api/architects` (collection wala, jo dono roles ko allowed hai) ko bhi **nigal jaayega** — aur User ko 403 milega jahan 200 milna chahiye tha. Isiliye poore config me single `*` hi use hua hai.

Trap 3 — `/error` ko permitAll kyun kiya

Jab 403 hota hai, servlet container ek internal **ERROR dispatch** karta hai `/error` pe — aur wo dispatch dobara security chain se guzarta hai.

Agar `/error` protected rehta, to wo internal request bhi block hoti → entry point chalta → **401 tumhare 403 ko overwrite kar deta**. Code me ye comment already likha hai. Ek line hatao, aur 11 role tests tootne lagenge — wo bhi aise reason se jo bilkul samajh nahi aayega.

Pura role matrix

Endpoint	Method	Admin	User
`/api/register` , `/api/login`	POST	**201 / 200**	**201 / 200**
`/api/architects`	POST	**201**	**403**
`/api/architects/{id}`	GET	**200**	**403**
`/api/architects`	GET	**200**	**200 ← dono**
`/api/architects/{id}`	PUT	**200**	**403**
`/api/inquiries`	POST	**403**	**201**
`/api/inquiries`	GET	**200**	**403**
`/api/inquiries/user/{id}`	GET	**403**	**200**
`/api/inquiries/{id}`	PUT	**200**	**403**
`/api/feedback`	POST	**403**	**201**
`/api/feedback`	GET	**200**	**403**
`/api/feedback/user/{id}`	GET	**403**	**200**

> **Notice — roles resource ke hisaab se ULTE ho jaate hain**
>
> **Architects** pe Admin ka control hai (wo architects manage karta hai).
> **Inquiries & Feedback** *banane* ka haq User ka hai (wo inquiry bhejta hai), par *sab dekhne* ka haq Admin ka.
> Ye inversion hi 11 tests ka core hai — isliye matrix yaad rakhna, logic se guess mat karna.

PART 4 — EXAMPLES

3 Examples: Request ka Pura Safar

> Ab sab files ko jod ke dekhte hain — ek successful request, ek 403, aur ek 401. Yehi teen cases 99% debugging cover kar lete hain.

Example 1 — Admin architect add karta hai (201 Created)

```
POST http://localhost:8080/api/architects
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...   ← Admin ka token
Content-Type: application/json

{ "name": "John Smith", "expertise": "Residential", "status": "Active" }
```

1. Request server pe aayi. `JwtAuthenticationFilter` sabse pehle chala (kyunki `addFilterBefore` se wire kiya hai).
2. Header me `"Bearer "` mila → token nikaala → `JwtUtils.validateToken()` → signature sahi, expire nahi hua → **true**
3. `extractUsername()` se email mila: `admin@test.com`
4. `MyUserDetailsService` ne DB se user uthaya → `UserPrinciple` me wrap → authority bani: `ROLE_Admin`
5. `SecurityContext` set ho gaya. **Authentication ✓ complete.**
6. `SecurityConfig` ka rule check: `POST /api/architects → hasRole("Admin")` → needs `ROLE_Admin` → user ke paas hai → **Authorization ✓**
7. **Request `ArchitectController` tak pahunchi → service → repo → DB insert → 201 Created + Architect JSON wapas.**

Example 2 — User wahi kaam karta hai (403 Forbidden)

```
POST http://localhost:8080/api/architects
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...   ← ab User ka token (valid hai!)
```

1. Steps 1–5 **bilkul same** chalte hain — token valid hai, DB me user hai, `SecurityContext` set ho jaata hai. Authority banti hai: `ROLE_User`

2. `SecurityConfig` rule: `hasRole("Admin")` chahiye → user ke paas `ROLE_User` hai → **match nahi hua**

3. `AccessDeniedHandler` trigger → **403 Forbidden**. Controller tak request pahunchi hi nahi.

> **Yahan 403 aata hai, 401 nahi — kyun?**
>
> Kyunki **authentication successful hui thi** (token valid tha, banda pehchana gaya). Sirf **authorization fail hui** (us bande ko is kaam ki permission nahi). Gate pe ID card chal gaya, par Floor 7 ka access nahi tha.

Example 3 — Koi bina token ke aata hai (401 Unauthorized)

```
GET http://localhost:8080/api/architects
// koi Authorization header nahi
```

1. `JwtAuthenticationFilter` chala — `header == null` → pura `if` block skip. `SecurityContext` **khaali** rehta hai.

2. Par filter fir bhi `filterChain.doFilter()` call karta hai — wo kabhi block nahi karta, sirf identity set karta hai (ya nahi karta).

3. `SecurityConfig`: `GET /api/architects → .authenticated()` → par koi authentication hai hi nahi → **fail**

4. `JwtAuthenticationEntryPoint.commence()` trigger → `sendError(401, "Unauthorized")` → **401**

Yahi response tumne Thunder Client me dekha tha:

```
{ "timestamp": "2026-10-04T06:14:09.087+00:00",
  "status": 401, "error": "Unauthorized",
  "message": "Unauthorized", "path": "/api/user/11" }
```

Teeno ka comparison

	Example 1	Example 2	Example 3
Token	Admin ka, valid	User ka, valid	Koi nahi
SecurityContext	**Set (ROLE_Admin)**	**Set (ROLE_User)**	**Khaali**
Authentication	**✓ Pass**	**✓ Pass**	**✗ Fail**
Authorization	**✓ Pass**	**✗ Fail**	— (chala hi nahi)
Controller pahuncha?	**Haan**	**Nahi**	**Nahi**
Response	**201**	**403**	**401**

PART 5 — REFERENCE

Debugging Cheat Sheet & Golden Rules

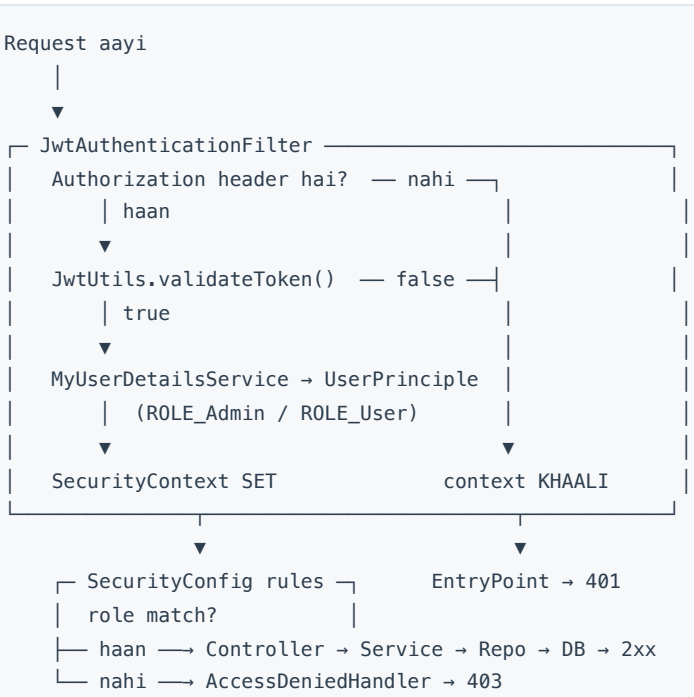
Symptom → Kahan dekhna hai

Symptom	Asli wajah	Kahan fix karo
Har jagah **401** aa raha, 403 ki jagah	Filter `SecurityContext` set nahi kar pa raha	`JwtAuthenticationFilter` — token parse / DB lookup fail ho raha hoga
Sahi role hone pe bhi **403**	`ROLE_` prefix missing, ya case mismatch	`UserPrinciple.getAuthorities()` — `"ROLE_" + role` check karo
User ko `GET /api/architects` pe 403	`**` wildcard ne collection rule nigal liya	`SecurityConfig` — `**` ko `*` karo
403 expect tha, 401 mila	`/error` permitAll nahi hai	`SecurityConfig` — `.requestMatchers("/error").permitAll()`
Login 200 deta hai par baad ke tests fail	Dono tokens me same role ja raha hai	`AuthController.login()` — `user.getUserRole()` DB se aa raha hai?
Browser me CORS error, Thunder Client me sab theek	CORS browser-only protection hai	`CrosConfig` + controllers ka `@CrossOrigin` origin
POST/PUT pe 403, GET theek	CSRF enabled reh gaya	`SecurityConfig` — `.csrf(csrf -> csrf.disable())`

Golden rules — ye kabhi mat todna

1. **Security disable mat karna.** SRS (p.43) kehta hai auto-evaluation ke liye security comment out kar do ya GET endpoints `permitAll()` kar do — **ye advice graded suite ko tod degi.** 11 tests real role enforcement pe depend karte hain. Ignore karo.
2. **Roles sirf URL matchers se enforce karo** (`SecurityConfig` me) — `@PreAuthorize` ke saath mix mat karo. Dono mila doge to conflicting behaviour milega jo debug karna mushkil hai.
3. **Single `*` hi use karo**, `**` kabhi nahi.
4. **`/user/*` rules hamesha broader rules se pehle** likho.
5. **Spring Boot 3 ⇒ `jakarta.*`** , `javax.*` nahi. Security 6 ⇒ `SecurityFilterChain` bean + lambda DSL; `WebSecurityConfigurerAdapter` ab exist hi nahi karta.
6. **Login response me key bilkul `token` honi chahiye** — 11 tests `body.get("token")` padhte hain.

Poora picture — ek nazar me



Ek line me pura system

Filter batata hai "tum kaun ho" (identity set karta hai), **Config** batata hai "tum kya kar sakte ho" (rules check karta hai). Identity nahi mili → 401. Identity mili par permission nahi → 403. Dono mile → controller chalta hai.